Crew Ops Advisor

Architecture, node-by-node flow, and where the line between the language model and deterministic code is drawn. dCortex hackathon submission.

1. The one rule that decides everything

> **The language model plans and explains. It never produces a fact and never does arithmetic.**

> A LangGraph agent decides *which* computations to run, in what order, and how to phrase the result for a controller under time pressure. Deterministic code does the computing. A verifier then checks every number, identifier, rule id, station code and date in the drafted reply against what the tools actually returned, and rejects anything unattested.

The problem statement puts 20% of the score on exactly this question, and says the obvious approach (put the data in the prompt, let the model answer) fails at Tier 2 and 3, because legality is exact arithmetic against a rulebook and an approximated duty-hour calculation is fluent, confident and wrong.

How the boundary is enforced rather than promised

A claim enforced only by a prompt is not enforced. `tests/test_boundary.py` walks the **import graph** of `contracts/`, `domain/`, `rules/`, `ops/`, `store/`, `tools/` and `verify/` and fails the build if any model client is reachable from any of them at any depth. It also asserts the mirror: that `agent/` *does* import one, so the boundary is not decorative. A third test re-parses every shipped dataset file, so a run that corrupted the answer keys fails loudly.

2. Physical shape

```
Next.js UI  --SSE-->  FastAPI  -->  LangGraph agent
                                          |
                            +-------------+-------------+
                            |             |             |
                         planner        tools        verifier
                          (LLM)    (deterministic)  (deterministic)
                                          |
                                +---------+---------+
                                rules engine     ops engine
                                (the 7 rules)   (cover search, costing,
```

```
                                        ranking, simulation)
                                              |
                                         WorldState
                                   (typed, immutable, loaded once)
                                              |
                                      SQLite projection
                                   (indexed, rebuilt at startup)
```

Path	Responsibility	Model allowed
data/	Provided dataset. Immutable, never regenerated.	no
contracts/	Shared types every layer imports. The seam.	no
domain/	Typed records, loader, immutable WorldState	no
rules/	Clock arithmetic, the seven rules, RuleTrace	never
ops/	Cover search, positioning, costing, ranking, simulation	never
store/	Indexed SQLite projection of the world	no
tools/	The 24 tools the agent calls. Wraps the above.	no
resolve/	Offline path: triage, intents, deterministic render	no
agent/	LangGraph graph, prompts, providers, memory, guards	yes
verify/	The grounding verifier	no
eval/	Scorecard against the shipped answer keys	no
web/	Next.js front end. No answering logic, ever.	no

3. Two answer paths, one Reply type

Every interface renders one `Reply`. Both paths use the same tools, the same guards and the same verifier; they differ only in who chooses the tools.

	Deterministic path	Agent path
Chooses tools by	regex intent match over 20 shapes	a language model, iteratively
Needs a key	no	yes
Latency	3 to 113 ms	6 to 30 s
Handles typos, aliases	only what normalisation covers	yes, natively
Reproducible	exactly	no (see section 11)

4. The LangGraph agent, node by node

```
START
  |
```

```
  v
route ------------- out of scope / greeting -------------> abstain --> END
  |
  v
plan            structured output, one retry on an empty plan
  |
  v
agent <---------------------+
  |  |                      |
  |  +--- tool calls ---> tools    (executes against ToolSurface,
  |                                 accumulates envelopes in state)
  v
verify
  |
  +-- verified -------------------------------------------------> END
  |
  +-- unattested atoms ------> repair --------> agent
  |
  +-- guard failure ---------> policy_repair -> agent
  |
  +-- second failure --------> abstain --------> END
```

route

Deterministic triage, costs nothing. Calls `canonical_question()` then `triage_question()`. Three outcomes: **greeting** (answered with a capability statement, no "I cannot"), **out of scope** (refused in ~2ms before any model spend, e.g. "what is the capital of France"), or **in scope** with a tier floor. Deliberately asymmetric: it refuses only when confident, because a wrong refusal here is unrecoverable while a wrong forward is caught downstream.

plan

One model call, no tools bound, structured output into `TurnPlan{intent, tier, steps}`. Emitted as the `plan` stream event so a controller watches the system decide before it acts. The model may *raise* the tier the deterministic classifier set, never lower it.

Retry on empty. `with_structured_output` does not raise when the model declines to emit the forced tool call: it returns `None`. Measured on deepseek-v4-flash that is four calls in six. So the node retries once and then emits a visible note. Before this, roughly half of all turns ran with no plan and degraded silently.

agent

The tool-calling loop. The model chooses tools and arguments and eventually writes prose. Capped at 8 iterations and a 30 second wall clock, checked before every model call and every tool batch.

tools

Not LangGraph's `ToolNode`. A custom node, so it can time each call, coerce arguments through the Pydantic arg model, accumulate every `ToolEnvelope` into typed state where the verifier can see the complete fact set, and emit progress events. It also **suppresses duplicate calls**: a call whose (tool, arguments) already ran this turn gets the earlier envelope back, keyed so that argument order is not a difference. The reply is still emitted, because a tool call with no matching result breaks the message sequence.

verify

Structural guards first, then the grounding check. Both deterministic. Details in sections 7 and 8.

repair / policy_repair

Exactly one correction pass, shared between them. `repair` is told precisely which atoms were unattested; `policy_repair` is told which guard failed and which tool would fix it. "Call `check_legality` for C-3310 on P-2291" is actionable; "be more careful" is not.

abstain

Builds a refusal a controller can act on: what was missing, what was established, and what the system *can* answer.

5. The deterministic path, step by step

```
question
  |
  v
canonical_question()      C1042 -> C-1042, bangalore -> BLR, DX 412 -> DX412
  |                       (the controller's own words are kept for Reply.question)
  v
triage_question()         greeting? out of scope? which tier?
  |
  v
```

```
match_intent()            20 intents, highest priority wins
  |
  v
intent.missing(entities)  a gap here becomes a clarification, not a guess
  |
  v
intent.build(entities, snapshot) -> list[PlannedCall]
  |
  v
run each PlannedCall against the ToolSurface -> envelopes
  |
  v
render(template, envelopes)   templates, never free text
  |
  v
run_guards() then Verifier.check()    identical to the agent path
  |
  v
Reply(mode=DETERMINISTIC)
```

An `Intent` carries: match patterns, a priority, required entities, a `build` that returns the tool plan, and a render template. The plan is where most of this project's remaining bugs lived: the tools and the rules engine were nearly always right, and the plan asked the wrong question.

Worked example: a sick call

```
"Captain C-3231 calls in sick at 01:30Z on 16 Sep for pairing P-2224."


triage      in scope, tier 2, crew=[C-3231], pairings=[P-2224], date=2026-09-16
intent      sick_impact (priority 75)
build       [ simulate_absence(crew_id=C-3231, from_date=2026-09-16),
              find_cover_options(for_crew_id=C-3231, pairing_id=P-2224,
                                 on_date=2026-09-16, include_rejected=True) ]
run         2 envelopes, 47 ms
render      impact template + structured Recommendation
verify      every atom attested
Reply       4 uncovered legs, 6 legal options ranked by cost, 19 exclusions
            each carrying the rule that excluded it
```

The second call is the whole fix for three of the six worked scenarios. A sick call is a

situation, not a question: a controller told a captain is sick needs to know who can take the pairing, even though the sentence never says "what should I do".

6. The tool surface

24 tools, one protocol (`contracts/tools.py`), one registry (`tools/registry.py`). The contract is the seam three layers build against, and `test_boundary.py` makes drift between the protocol, the registry and the agent's argument models a build failure, because Pydantic drops unknown fields silently: an argument missing from a ToolSpec is not rejected, it evaporates.

```
Class            Tools
--------------   ------------------------------------------------------------------------------
Retrieval (14)   find_crew, get_crew_detail, find_flights, find_pairings, get_duty_clocks,
                 list_reserves, find_expiring_certifications, get_pairing, get_roster,
                 find_crew_at_risk, aggregate, get_cost_rates, get_world_summary, explain_rule
Consequence      check_legality, simulate_absence, simulate_reassignment,
                 simulate_station_closure, simulate_delay, scan_duty_headroom, earliest_report
Recommendation   find_cover_options, plan_joint_cover, draft_notification
```

Retrieval-only is a real classification, not documentation: `tier_guard` refuses a Tier 2 or 3 answer whose tool calls were *all* retrieval. Retrieval establishes what is; it does not establish what follows.

Every tool returns the same envelope

```
ToolEnvelope{
  tool, args, ok, error,
  payload,              the structured result
  facts:  [Fact],       typed values, each with a derivation
  trace:  [TraceStep],  what the tool did, in order
  citations: [Citation] file and pointer into the dataset
}
```

`Fact.derivation` is mandatory on a computed fact and states the arithmetic rather than summarising it: `"11.25h duty + 1.5h delay = 12.75h"`. That string is what a controller reads to challenge the number, and it is also re-scanned by the verifier so the operands inside it are attested too.

7. The seven rules

```
Rule           Constraint                                          Engine method
```

```
------------   ------------------------------------------------------------   --------------------
RULE-FDP-01    Max flight duty period 13h, reduced by sectors flown           check_fdp
RULE-DUTY-02   Max 60 duty hours in any 7 consecutive days                    check_duty_window
RULE-FLT-03    Max 100 flight hours in any 28 consecutive days                check_flight_window
RULE-REST-04   Min 12 hours rest before commencing duty                       check_rest
RULE-QUAL-05   Valid rating for the assigned aircraft type                    check_qualification
RULE-CERT-06   All certifications valid on the duty date                      check_certifications
RULE-BASE-07   Reserve callout from base only, unless deadhead cost           check_base
               applied
```

Every check returns a `RuleTrace`: `rule_id, title, verdict, duty_date, limit, observed, unit, margin, margin_human, arithmetic, inputs, note`. A multi-day pairing returns one `DayLegality` per day and the overall verdict is the **worst** day. Legal on day one and breaching on day two is not a legal option.

8. The verifier

Deterministic: no model call, no network, no clock, no randomness. Same input, same report, forever.

```
1  extract   scan the prose for checkable atoms. Overlaps resolved by
             earliest start, then longest span, so "INR 18,500" is one
             currency atom and not also the integers 18 and 500.
             Typographic characters are ASCII folded first (see below).

2  attest    build the attested set from this turn's envelopes. Two
             channels, counted separately: the typed Fact channel is
             primary, the payload channel is a fallback over the same
             deterministic output.

3  compare   per kind. Durations to whole minutes rounded half up, so
             61.33h == 61h20m. Other numbers to two decimals.
             Identifiers, stations, rule ids and dates compare EXACTLY.

4  report    VERIFIED, REPAIRED, REJECTED or SKIPPED, naming every
             unattested atom and the sentence it came from.
```

> **A guard that rejects good answers is exactly as useless as one that passes bad ones.**

> Language models write typographically: `gpt-oss` renders a crew id as `C-3310` with U+2011 NON-BREAKING HYPHEN so it will not wrap. The extractor scanned for an ASCII hyphen, did not find one, and fell

> through to the bare integer `3310`. A number in no tool output cannot be attested, so the grounding check **rejected answers that were correct**. It cost four of sixteen Tier 1 questions and read exactly like the model being stupid. The fold is one character to one character, so span offsets stay valid, and a test pins the other direction: it may only recover an identifier that was genuinely written, never manufacture one from prose.

What the verifier cannot catch

- **Relations between attested atoms.** Every atom in "C-3310 is legal for P-2291" can be attested while the sentence is false, because a verdict is a relation between values, not a value. This is what the guards exist for.
- **Spelled-out numbers above twelve.** The prompt asks for digits; the guard does not enforce it.
- **Unit swaps on an attested value.** If 61.33 is attested, "61.33 minutes" passes. Catching it produces false rejections on correct answers, which is worse.

9. The guards

The verifier checks *values*. The guards ask a different question: was this answer *entitled to exist*? They run before the grounding check, and the first failure wins.

Guard	Refuses	Because
substance	An empty answer, or a bare refusal with no reason	A refusal has to name what was missing
breach_agreement	An answer that reaches an all-clear before mentioning a breach the tools computed	The worst output the system can produce
verdict	A legality claim with no rules-engine envelope this turn	A verdict is computed, never inferred
ranking	A ranked recommendation with no cover search	Ranking needs every candidate enumerated, rule checked and priced, including the rejected ones
tier	A Tier 2 or 3 answer built on retrieval alone	Retrieval does not establish what follows

Why breach_agreement exists

Q20 asks whether a 90 minute delay pushes the rostered crew over a limit. `simulate_delay` computed it correctly: breach, 12.75h against a 12.0h FDP limit. Six `check_legality` calls on the pairing *as scheduled* returned pass, because as scheduled it does pass. The model then led with **"the pairing passes all seven rules"** and corrected itself four sentences later. A controller reads the first line and acts on it, so that answer

dispatches a crew into an FDP breach.

Neither existing mechanism could see it: every value in that headline was real, and a rules tool *had* run. The guard checks **ordering**, not presence, so an answer that opens with the breach and then notes what passes still gets through. The deeper fix was to the tool: `simulate_delay` now emits its FDP evaluation as a `RuleTrace`, so the structured verdict agrees with the prose.

10. Provider layer and memory

`agent/providers.py` is the only module in the repository that names a model vendor. The graph is written against `BaseChatModel`. Precedence is anthropic, then openai, then ollama, so adding a hosted key switches everything over with no config edit. Three vendor quirks live there so the graph stays neutral: the default model id must follow the provider, sampling must be pinned per vendor (Ollama defaults to temperature 0.8), and `with_structured_output` has no portable method.

`agent/memory.py` keeps two stores in one SQLite file: LangGraph's checkpointer for message state, and a `turns` table holding every settled `Reply` as JSON for audit. Operational data lives in a separate store entirely, so "chat memory never feeds operational reasoning" is structural rather than a convention.

11. Measured results

Scored against the shipped answer keys in `questions.json` and `scenarios.json` by `eval/scorecard.py`. Accuracy excludes abstentions: it is the share of questions the system chose to answer that it got right.

Deterministic path, no model, all 38 questions

```
Tier   n    correct   abstained   wrong   accuracy   avg / p95

----   --   -------   ---------   -----   --------   ----------

1      16   15        1           0       100%       3 / 3 ms

2      14   9         5           0       100%       15 / 81 ms

3      8    2         6           0       100%       27 / 79 ms

All    38   26        12          0       100%       10 / 79 ms
```

The six worked scenarios, deterministic

```
Scenario   Result          What it exercises

--------   -------------   ------------------------------------------

S1         correct, 100%   Sick call, cover options, exclusions
```

```
S2          correct, 100%   Two-day pairing sick call. The flagship.
S3          correct, 100%   Station closure, 13 flights, 6 pairings
S4          partial, 70%    Technical delay, FDP breach
S5          correct, 100%   Expired certificate, illegal duty, recovery
S6          partial, 79%    Two simultaneous sick calls, joint allocation
```

Four correct, two partial, nothing wrong, all six grounded.

Agent path, Tier 1, four models compared

Model	correct	wrong	grounded	avg	p95
deepseek-v4-flash	16/16	0	16/16	6.5 s	12.1 s
gpt-oss:120b	13/16	1	15/16	7.2 s	10.0 s
glm-5.1	13/16	0	13/16	16.9 s	27.4 s
kimi-k2.6	5/16	0	5/16	37.3 s	63.2 s
qwen2.5:7b	unusable: returns an empty tool_calls list for a bound schema				

> **Read the accuracy column next to coverage, never alone.** `kimi-k2.6` scores **100% accuracy** on Tier 1: it declined 11 of 16 questions and got the other 5 right. Correct whenever it answers, and useless at a desk.

Reproducibility

These models do not honour temperature 0. Tier 2 on the agent path has scored 9, 10, 10, 12 and 13 across five runs of near-identical code. That spread is wider than most of the improvements measured against it, so agent-mode figures are quoted as ranges and single runs are not treated as evidence.

12. Known limitations

- **S4 and S6 remain partial.** S4 does not name the delayed flight in prose; S6 needs richer per-gap enumeration. Neither is wrong, both are incomplete.
- **Tier 3 answers 2 of 8 questions.** Coverage, not correctness: the six declined are declined honestly.
- **Thread memory is unproven.** The checkpoints table has zero rows, because every logged turn so far ran deterministic, which bypasses the graph. Multi-turn follow-up is implemented and undemonstrated.
- **The offline path matches fixed shapes.** Normalisation covers identifier spelling and station aliases; a question phrased in a way no pattern anticipates is declined rather than guessed.
- **Agent-mode latency.** Tier 2 p95 sits near the 30 second budget. The problem statement's line is 45 seconds.

13. What was actually hard

Recorded because the failure analysis is worth more than the feature list, and because every one of these was found by reading output rather than by reasoning about the design.

- **The verifier rejected its own correct answers** over a non-breaking hyphen. It failed in the direction that looks responsible, so it read as the model being weak. Cost four Tier 1 questions.
- **A verdict inversion.** The breach was computed and never carried into the structured reply, so the Reply asserted "legal" for a question whose answer was a breach.
- **Three lists that were counted but never named.** Closure flights, closure pairings, and a recommendation's candidates. Each turned a correct computation into a wrong or partial answer.
- **The planner returned nothing on four calls in six**, silently, so half of all turns ran with no plan and the controller was shown fallback boilerplate.
- **Two hand-maintained lists that had to agree and nothing made them.** The guards' legality-bearing tools and the contract's `REQUIRED_FOR` had drifted; a newly added tool registered in one and not the other made a guard refuse an answer the rules engine had produced. Now derived, with a test.

Every figure in this document came from `make test` and `python -m crewops.eval.scorecard` on the commit that ships it, not from memory.